

Home

简介

该项目的目标：

- 为修改 Minecraft ，使其使用自定义 Yggdrasil 服务提供工具
- 为自定义 Yggdrasil 服务端、使用自定义 Yggdrasil 服务的启动器提供技术规范
- 为玩家提供统一的非 Mojang 游戏外登录体验
 - 玩家可以使用任意实现该规范的启动器，登录任意实现该规范的 Yggdrasil 服务

该项目会对所有 API 作出详细说明，并且还会定义一些不属于 Yggdrasil 的 API 。这样做是为了最简化指定 Yggdrasil 服务的流程：只需要填写 Yggdrasil 服务对应的 URL ，就可以使用它。

如果你是 Yggdrasil 服务端开发者或启动器开发者，或是对本项目感兴趣，请 Watch 本项目，以了解规范最新的发展

开发者交流 QQ 群：926979364，Telegram 群：[@authlib_injector](#)。欢迎启动器或皮肤站开发者加入。普通用户请勿加群，可能会听不懂。

相关项目

- [yggdrasil-mock](#)
 - Yggdrasil 服务端规范的参考实现，以及 Yggdrasil API 的测试用例
 - 基于此项目的 Yggdrasil 服务端演示站点：[auth-demo.yushi.moe](#)
- [BMCLAPI](#)
 - BMCLAPI 为 authlib-injector 下载提供了一个镜像
- [Yggdrasil API for Blessing Skin](#)
 - Blessing Skin 皮肤站的 Yggdrasil 插件
- [HMCL](#)
 - HMCL v3.x 支持 authlib-injector
- [BakaXL](#)
 - BakaXL 3.0 支持 authlib-injector
- [LaunchHelper](#)
 - 想在 Multicraft 面板上使用 authlib-injector，可以尝试此项目

推荐的公共验证服务器

- [LittleSkin](#)
- [Ely.by](#)

捐助

BMCLAPI 为 authlib-injector 提供了下载镜像站。如果您想要支持 authlib-injector 的开发，您可以 [捐助 BMCLAPI](#)。

Yggdrasil 服务端技术规范

概述

本文旨在为实现 Yggdrasil 服务端提供非官方的技术规范。

本规范中所描述的服务端行为不一定与 Mojang 服务端的相同。这客观上是因为 Mojang 的服务端是闭源的，我们只能推测其内部逻辑，而所作出的推测难免会与实际存在出入。但事实上只要客户端能够正确理解并处理服务端的响应，那么其行为是否与 Mojang 服务端的相同，也就无关紧要了。

基本约定

字符编码

本文中字符编码一律使用 UTF-8。

请求与响应格式

若无特殊说明，请求与响应均为 JSON 格式（如果有 body），Content-Type 均为 `application/json; charset=utf-8`。

所有 API 都应该使用 HTTPS 协议。

错误信息格式

```
{
  "error": "错误的简要描述（机器可读）",
  "errorMessage": "错误的详细信息（人类可读）",
  "cause": "该错误的原因（可选）"
}
```

当遇到本文中已说明的异常情况时，返回的错误信息应符合对应的要求。

下表列举了常见异常情况下的错误信息。除特殊说明外，`cause` 一般不包含。

非标准指由于无法使 Mojang 的 Yggdrasil 服务器触发对应异常，而只能推测该种情况下的错误信息。

未定义指该项并没有明确要求。

异常情况	HTTP 状态 码	Error	Error Message
一般 HTTP 异常 (非业务异常， 如 <i>Not Found</i> 、 <i>Method Not Allowed</i>)	未定 义	该 HTTP 状态对应的 <i>Reason Phrase</i> (于 HTTP/1.1 中定义)	未定义
令牌无效	403	ForbiddenOperationException	Invalid token.
密码错误，或短 时间内多次登录 失败而被暂时禁 止登录	403	ForbiddenOperationException	Invalid credentials. Invalid username or password.
试图向一个已经 绑定了角色的令 牌指定其要绑定 的角色	400	IllegalArgumentException	Access token already has a profile assigned.
试图向一个令牌 绑定不属于其对 应用户的角色 (非标准)	403	ForbiddenOperationException	未定义
试图使用一个错 误的角色加入服 务器	403	ForbiddenOperationException	Invalid token.

数据格式

我们约定以下数据格式

- 无符号 **UUID**: 指去掉所有 - 字符后的 UUID 字符串

模型

用户

一个系统中可以存在若干个用户，用户具有以下属性：

- ID
- 邮箱
- 密码

其中 ID 为一个无符号 UUID。邮箱可以变更，但需要保证唯一。

用户信息的序列化

用户信息序列化后符合以下格式：

```
{
  "id":"用户的 ID",
  "properties":[ // 用户的属性（数组，每一元素为一个属性）
    { // 一项属性
      "name":"属性的名称",
      "value":"属性的值",
    }
    // ,...（可以有更多）
  ]
}
```

用户属性中目前已知的项目如下：

名称	值
preferredLanguage	(可选) 用户的偏好语言，例如 en 、 zh_CN

角色 (Profile)

Mojang 当前不支持多角色，不保证多角色部分内容的正确性。

角色与账号为多对一关系。一个角色对应 Minecraft 中的一个实体玩家。角色具有以下属性：

- UUID
- 名称
- 材质模型，可选值有： default 、 slim
 - **default**： 正常手臂宽度（4px）的皮肤

- **slim**: 细手臂 (3px) 的皮肤
- 材质
 - 类型为映射
 - key 可选值有: SKIN、CAPE
 - value 类型为 URL

UUID 和名称均为全局唯一, 但名称可变。应避免使用名称作为标识。

角色 UUID 的生成

若不考虑兼容性, 角色的 UUID 一般为随机生成 ([Version 4](#))。

但 Minecraft 仅使用 UUID 作为角色标识符, 不同 UUID 的角色即使名称相同也被认为是不同的。如果一个 Minecraft 服务器从其他登录系统 (正版验证、离线验证或其他) 迁移到本登录系统, 并且角色的 UUID 发生了变化, 则该角色的数据将丢失。为了避免这种情况, 必须保证对于同一个角色, 本系统生成的 UUID 与其在先前系统中的 UUID 是相同的。

兼容离线验证

若 Minecraft 服务器原先采用的是离线验证, 则角色 UUID 是角色名称的一元函数。如果 Yggdrasil 服务端使用此方法生成角色 UUID, 就可以实现与离线验证系统之间的双向兼容, 即可以在不丢失角色数据的情况下, 在离线验证系统和本登录系统之间切换。

从角色名称计算角色 UUID 的代码如下 (Java):

```
UUID.nameUUIDFromBytes(("OfflinePlayer:" +  
characterName).getBytes(StandardCharsets.UTF_8))
```

在其他语言中的实现:

- [PHP](#)

角色信息的序列化

角色信息序列化后符合以下格式:

```
{
  "id": "角色 UUID (无符号) ",
  "name": "角色名称",
  "properties": [ // 角色的属性 (数组, 每一元素为一个属性) (仅在特定情况下需要包含)
    { // 一项属性
      "name": "属性的名称",
      "value": "属性的值",
      "signature": "属性值的数字签名 (仅在特定情况下需要包含) "
    }
    // , ... (可以有更多)
  ]
}
```

角色属性 (`properties`) 及数字签名 (`signature`) 在无特殊说明的情况下不需要包含。

`signature` 是属性值的数字签名, 使用 Base64 编码。签名算法为 SHA1withRSA, 见 [PKCS #1](#)。关于签名密钥的详细介绍, 见 [签名密钥对](#)。

角色属性中可以包含以下项目:

名称	值
textures	(可选) Base64 编码的 JSON 字符串, 包含了角色的材质信息, 详见 § textures 材质信息属性 。
uploadableTextures	(可选) 该角色可以上传的材质类型, 为 authlib-injector 自行规定的属性, 详见 § uploadableTextures 可上传的材质类型

textures 材质信息属性

以下为材质信息的格式, 将这段 JSON 进行 Base64 编码后, 即为 `textures` 角色属性的值。

```
{
  "timestamp": 该属性值被生成时的时间戳（Java 时间戳格式，即自 1970-01-01 00:00:00 UTC
至今经过的毫秒数），
  "profileId": "角色 UUID（无符号）",
  "profileName": "角色名称",
  "textures": { // 角色的材质
    "材质类型（如 SKIN）": { // 若角色不具有该项材质，则不必包含
      "url": "材质的 URL",
      "metadata": { // 材质的元数据，若没有则不必包含
        "名称": "值"
        // ,...（可以有更多）
      }
    }
    // ,...（可以有更多）
  }
}
```

材质元数据中目前已知的项目有 `model`，其对应角色的材质模型，取值为 `default` 或 `slim`。

uploadableTextures 可上传的材质类型

注意： 这一角色属性是由 authlib-injector 文档规定的，Mojang 返回的角色属性是不包含这一项的。Mojang 仅允许用户上传皮肤，不允许上传披风。

考虑到并非所有验证服务器都允许用户上传皮肤和披风，因此 authlib-injector 规定了 `uploadableTextures` 角色属性，其表示角色可以上传的材质类型。

该属性的值是一个逗号分隔的列表，包含了可以上传的材质类型。材质类型目前有 `skin` 和 `cape` 两种。

例如，`uploadableTextures` 属性的值若为 `skin`，则表示可以为该角色上传皮肤，但不能上传披风；值若为 `skin,cape`，则既可以上传皮肤，又可以上传披风。

如果不存在 `uploadableTextures` 属性，则不能为该角色上传任何类型的材质。

关于材质上传接口的介绍，请参考 [S材质上传](#)。

材质 URL 规范

Minecraft 将材质 hash 作为材质的标识。每当客户端下载一个材质后，便会将其缓存在本地，以后若需要相同 hash 的材质，则会直接使用缓存。而这个 hash 并不是由客户端计算的。Yggdrasil 服务端应先计算好材质 hash，将其作为材质 URL 的文件名，即从 URL 最后一个 `/`（不包括）开始一直到结尾的这一段子串。而客户端会直接将 URL 的文件名作为材质的 hash。

例如下面这个 URL，它所代表的材质的 hash 为

e051c27e803ba15de78a1d1e83491411dfffb6d7fd2886da0a6c34a2161f7ca99：

```
https://yggdrasil.example.com/textures/e051c27e803ba15de78a1d1e83491411dfffb6d7fd2886da0a6c34a2161f7ca99
```

安全警告：

- 材质 URL 响应头中的 `Content-Type` 必须为 `image/png`。若未指定，则存在 MIME Sniffing Attack 的风险。

材质 hash 的计算方法服务端可自行选择，建议使用 SHA-256 或者更加安全的 hash 算法。作为参考，Mojang 官方的方法是计算图像文件的 SHA-256。

用户上传材质的安全性

安全警告：

- 若不对用户上传材质进行处理，则**可能导致远程代码执行**
- 在读取材质前，若不先检查图像大小，则**可导致拒绝服务攻击**

关于此安全缺陷的详细信息：[未经检查的用户上传材质可能导致远程代码执行 #10](#)

除了位图数据外，PNG 文件还可以存储其他数据。如果 Yggdrasil 服务端不对用户上传的材质进行检查，则攻击者可以在其中藏匿恶意代码，并通过 Yggdrasil 服务端分发到客户端。因此，Yggdrasil 服务端**必须**对用户上传的材质进行处理，除去其中任何与位图无关的数据。具体做法如下：

1. 读取该 PNG 文件中图像的大小，如果过大则应拒绝。
 - 即使是非常小的 PNG 文件也可以存储一幅足以消耗计算机所有内存的图像（即 PNG Bomb），因此切不可在检查图像大小前就将其完整读入。
2. 检查图像是否为合法的皮肤/披风材质。
 - 皮肤的宽高为 64x32 的整数倍或 64x64 的整数倍，披风的宽高为 64x32 的整数倍或 22x17 的整数倍。宽高为 22x17 整数倍的披风并非标准尺寸的披风，服务端需要用透明像素将其宽高补足至 64x32 的整数倍。
3. 将图像文件重新保存，去除无关的元数据。

实现提示：在 Java 中可使用 `ImageReader.getWidth()` 在不读入整个图像的情况下获取其尺寸。

令牌 (Token)

令牌与账号为多对一关系。令牌是一种登录凭证，具有时效性。令牌具有以下属性：

- accessToken
- clientToken
- 绑定的角色
- 颁发时间

其中 accessToken 和 clientToken 为任意字符串（可以是无符号 UUID 或 JWT）。accessToken 由服务端随机生成，clientToken 由客户端提供。

鉴于 accessToken 的随机性，它可以被作为主键。而 clientToken 不具有唯一性。

绑定的角色可以为空。它代表了能使用该令牌进行游戏的角色。

一个用户可以同时有多个令牌，但服务端也应该对令牌数量加以限制。当令牌数量超出限制（如 10 个）时，则应先吊销最旧的令牌，之后再颁发新的令牌。

令牌的状态

令牌有以下三种状态：

- **有效**
 - 处于该状态的令牌可以进行各项操作，如[进服验证](#)、[刷新](#)。
 - 新颁发的令牌（即通过[登录](#)、[刷新](#)颁发的令牌）处于该状态。
- **暂时失效**
 - 处于该状态的令牌除了进行[刷新操作](#)外，无权进行任何操作。
 - 当令牌绑定的角色改名后，令牌应被标记为**暂时失效**状态。
 - 这是为了让启动器刷新令牌，从而获取到新的角色名称。（见 [#40](#)）
 - *该状态并不是必须要实现的（详细介绍见下）。*
- **无效**
 - 处于该状态的令牌无权进行任何操作。
 - 令牌被吊销后处于该状态。这里的吊销包括[显式吊销](#)、[登出](#)、[刷新](#)后吊销原令牌、令牌过期。

令牌的状态只能由有效变为无效，或是由有效变为暂时失效再变为无效，这个过程是不可逆的。刷新操作仅颁发一个新的令牌，并不能使原令牌重新回到有效状态。

令牌应当有一个过期时限（如 15 天）。当自颁发起所经过的时间超过该时限，令牌过期。

关于暂时失效状态

Mojang 对暂时失效状态的实现是这样的：对启动器而言，若令牌处于暂时失效状态，则会刷新令牌，获得一个新的处于有效状态的令牌；对 Yggdrasil 服务端而言，仅最后颁发的令牌才是有效的，先前颁发的其它令牌都处于暂时失效状态。

Mojang 之所以这么做，可能是为了防止用户多地同时登录（仅使最后一个 session 有效）。但事实上，即使服务端没有实现暂时失效状态，启动器的逻辑也是可以正常工作的。

当然，就算我们要实现暂时失效状态，也并不需要以 Mojang 的实现为范本。只需要启动器能够正确处理，任何实现都是可以的。下面给出一个不同于 Mojang 的实现例子：

取一个短于令牌过期时限的时间段作为有效和暂时失效的分界点。若自颁发起经过的时间在该时限内，则令牌有效；若超过该时限，但仍在过期时限内，则令牌暂时失效。

这种做法实现了这样的功能：玩家如果经常进行登录操作，除了第一次登录就不需要输入密码了。而当他长时间未登录时则需要重新输入密码。

Yggdrasil API

用户部分

登录

POST /authserver/authenticate

使用密码进行身份验证，并分配一个新的令牌。

请求格式：

```
{
  "username": "邮箱（或其他凭证，详见 §使用角色名称登录）",
  "password": "密码",
  "clientToken": "由客户端指定的令牌的 clientToken（可选）",
  "requestUser": true/false, // 是否在响应中包含用户信息，默认 false
  "agent": {
    "name": "Minecraft",
    "version": 1
  }
}
```

若请求中未包含 `clientToken`，服务端应该随机生成一个无符号 UUID 作为 `clientToken`。但需要注意 `clientToken` 可以为任何字符串，即请求中提供任何 `clientToken` 都是可以接受的，不一定要为无符号 UUID。

对于令牌要绑定的角色：若用户没有任何角色，则为空；若用户仅有一个角色，那么通常绑定到该角色；若用户有多个角色，通常为 `空`，以便客户端进行选择。也就是说如果绑定的角色为 `空`，则需要客户端进行角色选择。

响应格式：

```
{
  "accessToken": "令牌的 accessToken",
  "clientToken": "令牌的 clientToken",
  "availableProfiles": [ // 用户可用角色列表
    // ... 每一项为一个角色（格式见 §角色信息的序列化）
  ],
  "selectedProfile": {
    // ... 绑定的角色，若为空，则不需要包含（格式见 §角色信息的序列化）
  },
  "user": {
    // ... 用户信息（仅当请求中 requestUser 为 true 时包含，格式见 §用户信息的序列化）
  }
}
```

安全提示： 该 API 可以被用于密码暴力破解，应受到速率限制。限制应针对用户，而不是客户端 IP。

使用角色名称登录

除使用邮箱登录外，验证服务器还可以允许用户使用角色名称登录。要实现这一点，验证服务器需要进行以下工作：

- 将 API 元数据中的 `feature.non_email_login` 字段设置为 `true`。（见 [API 元数据获取§功能选项](#)）
- 接受在[登录接口](#)中使用角色名称作为 `username` 参数。

当用户使用角色名称登录时，验证服务器应**自动将令牌绑定到相应角色**，即上文响应中的 `selectedProfile` 应为用户登录时所用的角色。

这种情况下，如果用户拥有多个角色，那么他可以省去选择角色的操作。考虑到某些程序不支持多角色（例如 Geyser），还可以通过上述方法绕过角色选择。

刷新

POST `/authserver/refresh`

吊销原令牌，并颁发一个新的令牌。

请求格式：

```
{
  "accessToken":"令牌的 accessToken",
  "clientToken":"令牌的 clientToken (可选) ",
  "requestUser":true/false, // 是否在响应中包含用户信息，默认 false
  "selectedProfile":{
    // ... 要选择的角色 (可选，格式见 $角色信息的序列化)
  }
}
```

当指定 `clientToken` 时，服务端应检查 `accessToken` 和 `clientToken` 是否有效，否则只需要检查 `accessToken`。

颁发的新令牌的 `clientToken` 应与原令牌的相同。

如果请求中包含 `selectedProfile`，那么这就是一个选择角色的操作。此操作要求原令牌所绑定的角色为空，而新令牌则将绑定到 `selectedProfile` 所指定的角色上。如果不包含 `selectedProfile`，那么新令牌所绑定的角色和原令牌相同。

刷新操作在令牌暂时失效时依然可以执行。若请求失败，原令牌依然有效。

响应格式：

```
{
  "accessToken":"新令牌的 accessToken",
  "clientToken":"新令牌的 clientToken",
  "selectedProfile":{
    // ... 新令牌绑定的角色，若为空，则不需要包含 (格式见 $角色信息的序列化)
  },
  "user":{
    // ... 用户信息 (仅当请求中 requestUser 为 true 时包含，格式见 $用户信息的序列化)
  }
}
```

验证令牌

POST /authserver/validate

检验令牌是否有效。

请求格式：

```
{  
  "accessToken": "令牌的 accessToken",  
  "clientToken": "令牌的 clientToken (可选)"  
}
```

当指定 `clientToken` 时，服务端应检查 `accessToken` 和 `clientToken` 是否有效，否则只需要检查 `accessToken`。

若令牌有效，服务端应返回 HTTP 状态 `204 No Content`，否则作为令牌无效的异常情况处理。

吊销令牌

POST /authserver/invalidate

吊销给定令牌。

请求格式：

```
{  
  "accessToken": "令牌的 accessToken",  
  "clientToken": "令牌的 clientToken (可选)"  
}
```

服务端只需要检查 `accessToken`，即无论 `clientToken` 为何值都不会造成影响。

无论操作是否成功，服务端应返回 HTTP 状态 `204 No Content`。

登出

POST /authserver/signout

吊销用户的所有令牌。

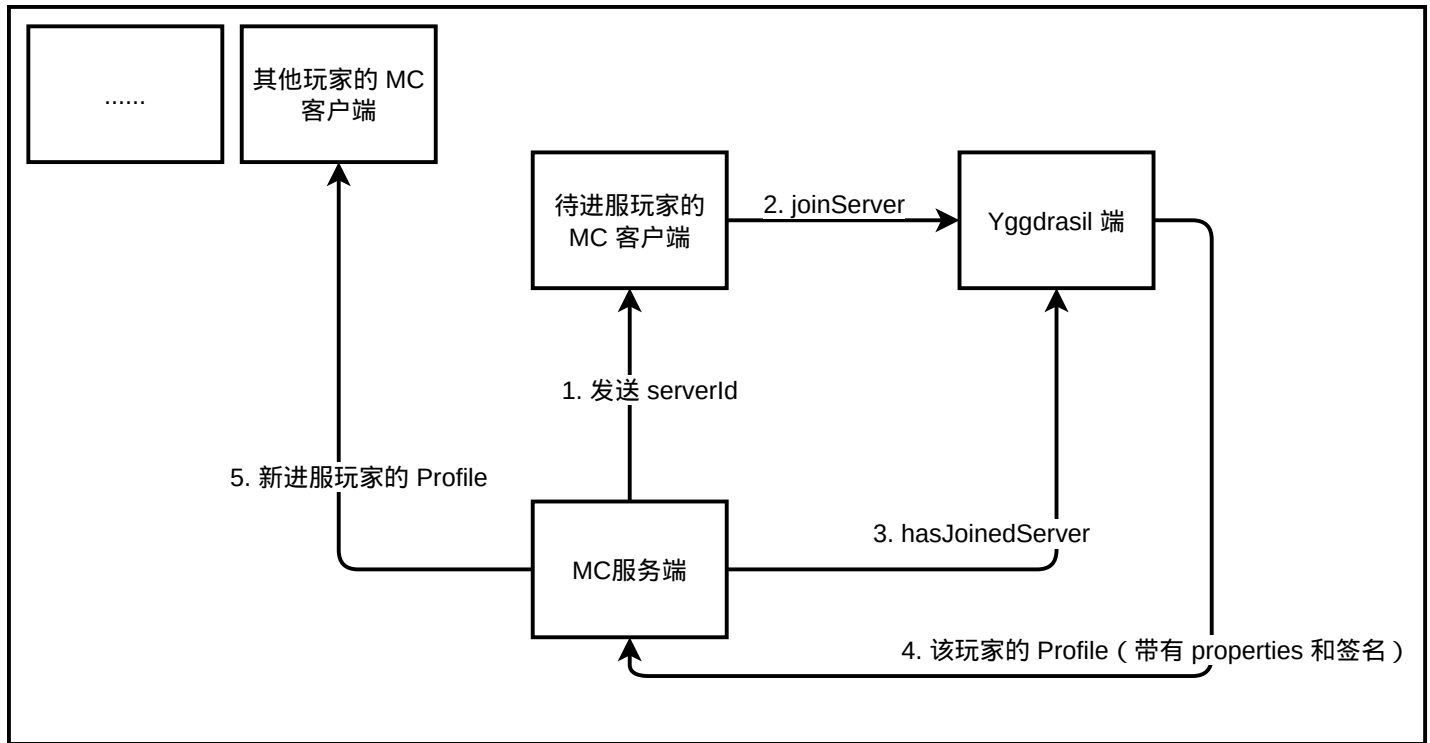
请求格式：

```
{  
  "username": "邮箱",  
  "password": "密码"  
}
```

若操作成功，服务端应返回 HTTP 状态 `204 No Content`。

安全提示： 该 API 也可用于判断密码的正确性，因此应受到和登录 API 一样的速率限制。

会话部分



1. 服务端向待进服的玩家发送 `serverId`，其可以被视为一个随机的字符串。
2. 客户端将该 `serverId` 连同自己的 `accessToken` 发送给 Yggdrasil 端，请求验证。
3. 服务端将 `serverId` 及玩家的名字发送给 Yggdrasil 端，查询玩家是否已完成验证。
4. 若玩家验证成功，则 Yggdrasil 端将玩家的 Profile 返回给服务端，其中包含材质信息和数字签名。
5. 服务端将该玩家的 Profile 分发给服内所有玩家。

上图使用 draw.io 绘制，源文件见 [mc入服原理.drawio](#)。

该部分用于角色进入服务器时的验证。主要流程如下：

1. **Minecraft 服务端**和 **Minecraft 客户端**共同生成一段字符串（`serverId`），其可以被认为是随机的
2. **Minecraft 客户端**将 `serverId` 及令牌发送给 **Yggdrasil 服务端**（要求令牌有效）
3. **Minecraft 服务端**请求 **Yggdrasil 服务端**检查客户端会话的有效性，即客户端是否成功进行第 2 步

客户端进入服务器

POST `/sessionserver/session/minecraft/join`

记录服务端发送给客户端的 `serverId`，以备服务端检查。

请求格式：

```
{
  "accessToken": "令牌的 accessToken",
  "selectedProfile": "该令牌绑定的角色的 UUID（无符号）",
  "serverId": "服务端发送给客户端的 serverId"
}
```

仅当 `accessToken` 有效，且 `selectedProfile` 与令牌所绑定的角色一致时，操作才成功。

服务端应记录以下信息：

- `serverId`
- `accessToken`
- 发送该请求的客户端 IP

实现时请注意：以上信息应记录在内存数据库中（如 Redis），且应该设置过期时间（如 30 秒）。鉴于 `serverId` 的随机性，可以将其作为主键。

若操作成功，服务端应返回 HTTP 状态 `204 No Content`。

服务端验证客户端

```
GET /sessionserver/session/minecraft/hasJoined?username={username}&serverId={serverId}&ip={ip}
```

检查客户端会话的有效性，即数据库中是否存在该 `serverId` 的记录，且信息正确。

请求参数：

参数	值
username	角色的名称
serverId	服务端发送给客户端的 serverId
ip (可选)	Minecraft 服务端获取到的客户端 IP，仅当 <code>prevent-proxy-connections</code> 选项开启时包含

`username` 需要与 `serverId` 所对应令牌所绑定的角色的名称相同。

响应格式：

```
{
    // ... 令牌所绑定角色的完整信息（包含角色属性及数字签名，格式见 §角色信息的序列化）
}
```

若操作失败，服务端应返回 HTTP 状态 204 No Content。

角色部分

该部分用于角色信息的查询。

查询角色属性

```
GET /sessionserver/session/minecraft/profile/{uuid}?unsigned={unsigned}
```

查询指定角色的完整信息（包含角色属性）。

请求参数：

参数	值
uuid	角色的 UUID（无符号）
unsigned <i>（可选）</i>	true 或 false。是否在响应中 不包含 数字签名，默认为 true

响应格式：

```
{
    // ... 角色信息（包含角色属性。若 unsigned 为 false，还需要包含数字签名。格式见 §角色信息的序列化）
}
```

若角色不存在，服务端应返回 HTTP 状态 204 No Content。

按名称批量查询角色

```
POST /api/profiles/minecraft
```

批量查询角色名称所对应的角色。

请求格式：

```
[
  "角色名称"
  // ,... 还可以有更多
]
```

服务端查询各个角色名称所对应的角色信息，并将其包含在响应中。不存在的角色不需要包含。响应中角色信息的先后次序无要求。

响应格式：

```
[
  {
    // 角色信息（注意：不包含角色属性。格式见 §角色信息的序列化）
  }
  // ,... （可以有更多）
]
```

安全提示： 为防止 CC 攻击，需要为单次查询的角色数目设置最大值，该值至少为 2。

材质上传

```
PUT /api/user/profile/{uuid}/{textureType}
DELETE /api/user/profile/{uuid}/{textureType}
```

设置或清除指定角色的材质。

并非所有角色都可以上传皮肤和披风。要获取当前角色能够上传的材质类型，参见 [§uploadableTextures](#) 可上传的材质类型。

请求参数：

参数	值
uuid	角色的 UUID（无符号）
textureType	材质类型，可以为 skin（皮肤）或 cape（披风）

请求需要带上 HTTP 头部 `Authorization: Bearer {accessToken}` 进行认证。若未包含 `Authorization` 头或 `accessToken` 无效，则返回 `401 Unauthorized`。

如果操作成功，则返回 `204 No Content`。

下面分别介绍 PUT 和 DELETE 这两个 HTTP 方法的用法：

PUT 上传材质

请求的 Content-Type 为 multipart/form-data ，请求载荷由以下部分组成：

名称 (name)	内容
model	(仅用于皮肤) 皮肤的材质模型，可以为 slim （细胳膊皮肤）或空字符串（普通皮肤）。
file	材质图像，Content-Type 须为 image/png 。 建议客户端设置 Content-Disposition 中的 filename 参数为材质图像的文件名，这可以被验证服务器用作材质的备注。

如果操作成功，则返回 204 No Content 。

DELETE 清除材质

清除材质后，该类型的材质将恢复为默认。

扩展 API

以下 API 是为了方便 authlib-injector 进行自动配置而设计的。

API 元数据获取

GET /

响应格式：

```
{
  "meta":{
    // 服务端的元数据，内容任意
  },
  "skinDomains":[ // 材质域名白名单
    "域名匹配规则 1"
    // ,...
  ],
  "signaturePublickey":"用于验证数字签名的公钥"
}
```

`signaturePublicKey` 是 PEM 格式的公钥，用于验证角色属性的数字签名。其以 `-----BEGIN PUBLIC KEY-----` 开头，以 `-----END PUBLIC KEY-----` 结尾，中间允许出现换行符，但不允许出现其他空白字符（亦允许文末出现换行符）。

材质域名白名单

Minecraft 仅会从白名单中的域名下载材质。如果材质 URL 的域名不在白名单中，则会出现 `Textures payload has been tampered with (non-whitelisted domain)` 错误。采用此机制的原因见 [MC-78491](#)。

材质白名单默认包含 `.minecraft.net`、`.mojang.com` 两项规则，你可以设置 `skinDomains` 属性以添加额外的白名单规则。规则格式如下：

- 如果规则以 `.`（dot）开头，则匹配以这一规则结尾的域名。
 - 例如 `.example.com` 匹配 `a.example.com`、`b.a.example.com`，**不匹配** `example.com`。
- 如果规则**不以** `.`（dot）开头，则匹配的域名须与规则**完全相同**。
 - 例如 `example.com` 匹配 `example.com`，**不匹配** `a.example.com`、`eexample.com`。

meta 中的元数据

`meta` 中的内容没有强制要求，以下字段均为可选。

服务端基本信息

Key	Value
<code>serverName</code>	服务器名称
<code>implementationName</code>	服务端实现的名称
<code>implementationVersion</code>	服务端实现的版本

服务器网址

如果您需要在启动器中展示验证服务器首页地址、注册页面地址等信息，您可以在 `meta` 中添加一个 `links` 字段。

`links` 字段的类型是对象，其中可以包含：

Key	Value
<code>homepage</code>	验证服务器首页地址
<code>register</code>	注册页面地址

功能选项

以下带有 **(advanced)** 标注的字段为高级选项，通常情况下**不需要**设置。

Key	Value
feature.non_email_login	布尔值，指示验证服务器是否支持使用邮箱之外的凭证登录（如角色名登录），默认为 false。 详情见 §使用角色名称登录 。
feature.legacy_skin_api	(advanced) 布尔值，指示验证服务器是否支持旧式皮肤 API，即 GET <code>/skins/MinecraftSkins/{username}.png</code> 。 当未指定或值为 false 时，authlib-injector 会使用内建的 HTTP 服务器在本地处理对该 API 的请求；若值为 true，请求将由验证服务器处理。 详情见 README § 参数 中的 - <code>Dauthlibinjector.legacySkinPolyfill</code> 选项。
feature.no_mojang_namespace	(advanced) 布尔值，是否禁用 authlib-injector 的 Mojang 命名空间（@mojang 后缀）功能，默认为 false。 详情见 README § 参数 中的 - <code>Dauthlibinjector.mojangNamespace</code> 选项。
feature.enable_mojang_anti_features	(advanced) 布尔值，是否开启 Minecraft 的 anti-features，默认为 false。 详情见 README § 参数 中的 - <code>Dauthlibinjector.mojangAntiFeatures</code> 选项。
feature.enable_profile_key	(advanced) 布尔值，指示验证服务器是否支持 Minecraft 的消息签名密钥对功能（即多人游戏中聊天消息的数字签名），默认为 false。 开启后，Minecraft 将通过 POST <code>/minecraftservices/player/certificates</code> 这一 API 获取验证服务器颁发的密钥对。当未指定或为 false 时，Minecraft 将不会在聊天消息中包含数字签名。 详情见 README § 参数 中的 - <code>Dauthlibinjector.profileKey</code> 选项。
feature.username_check	(advanced) 布尔值，指示 authlib-injector 是否启用用户名验证功能，默认为 false。 开启后用户名中带有特殊字符的玩家将无法加入

Key	Value
	服务器，详情见 README § 参数 中的 <code>- Dauthlibinjector.usernameCheck</code> 选项。

响应示例

```
{
  "meta": {
    "implementationName": "yggdrasil-mock-server",
    "implementationVersion": "0.0.1",
    "serverName": "yushijinhun's Example Authentication Server",
    "links": {
      "homepage": "https://skin.example.com/",
      "register": "https://skin.example.com/register"
    },
    "feature.non_email_login": true
  },
  "skinDomains": [
    "example.com",
    ".example.com"
  ],
  "signaturePublickey": "-----BEGIN PUBLIC KEY-----\nMIICIj... (省略) ...EAAQ==\n-----END PUBLIC KEY-----\n"
}
```

API 地址指示（ALI）

API 地址指示（API Location Indication，简称 ALI）是一个 HTTP 响应头字段 `X-Authlib-Injector-API-Location`，起到服务发现的作用。ALI 的值为相对 URL 或绝对 URL，它指向与当前页面相关联的 Yggdrasil API。

使用 ALI 后，用户只需输入一个与 Yggdrasil API 相关联的地址即可，不必输入真正的 API 地址。例如，`https://skin.example.com/api/yggdrasil/` 可以被简化为 `skin.example.com`。支持 ALI 的启动器会请求 `(https://)skin.example.com`，识别响应中的 ALI 头字段，并根据它找到真正的 API 地址。

皮肤站可以在首页，或在全站启用 ALI。启用 ALI 的方法为在 HTTP 响应中添加 `X-Authlib-Injector-API-Location` 头字段，例如：

```
X-Authlib-Injector-API-Location: /api/yggdrasil/ # 使用相对 URL
X-Authlib-Injector-API-Location: https://skin.example.com/api/yggdrasil/ # 亦可使用绝对 URL，支持跨域
```

当一个页面的 ALI 指向其本身时，这个 ALI 会被忽略。

参见

- [Authentication - wiki.vg](#)
- [Mojang API - wiki.vg](#)
- [Protocol - wiki.vg](#)

参考实现

[yggdrasil-mock](#) 为本规范的参考实现。

启动器技术规范

概述

本文旨在为启动器实现 authlib-injector 规范提供技术指导。由于该功能需要调用 Yggdrasil API，因此建议您在阅读本文前，先阅读 [Yggdrasil 服务端技术规范](#)。

在启动器中，本登录方式可以被称为【外置登录（authlib-injector）】或【authlib-injector 登录】。我们推荐您使用含义更为明确的前者。

验证服务器

验证服务器（即 Yggdrasil 服务器）是整个验证系统的核心，所有验证相关的请求都将被发往它。

为了确定一个验证服务器，启动器应当存储该验证服务器的 API 地址（即 API Root，如 `https://example.com/api/yggdrasil/`）。

启动器可以仅支持一个验证服务器，也可以支持多个验证服务器。支持多验证服务器就意味着，启动器中可以同时存在多个账户，并且这些账户可以属于不同的验证服务器。

验证服务器的设置

设置验证服务器的操作一般由玩家完成，但也存在着服主进行设置、配置文件同启动器和游戏一起分发的情况。下面介绍几种设置验证服务器的途径：

在配置文件中指定

启动器可以将 API 地址直接存储于配置文件中，让用户通过修改配置文件的方式设置验证服务器。这种配置方式实现简单，如果你的启动器只用作服务器专用启动器，则可以使用这种配置方式。

在启动器中输入地址

在这种配置方式下，用户通过在启动器中输入 URL 来完成验证服务器的设置。这里的 URL 可能是完整的 API 地址（如 `https://example.com/api/yggdrasil/`），也可能是缩略的地址（如 `example.com`）。

当 URL 未标明协议（HTTPS 或 HTTP）时，我们约定将其自动补全为 **HTTPS** 协议。亦即，`example.com/api/yggdrasil/` 应当被理解为 `https://example.com/api/yggdrasil/`。

出于安全考虑，启动器即使无法通过 HTTPS 协议连接，也**不得**降级到明文的 HTTP 协议。

同时，authlib-injector 规定了一种服务发现机制，称为 API 地址指示（ALI）。它用于将用户输入的缩略的、不完整的地址，转换为完整的 API 地址。

处理 API 地址指示（ALI）

为了将用户输入的地址解析为真正的 API 地址，启动器需要进行如下操作：

1. 如果 URL 缺少协议，则将其补全为 HTTPS 协议。
2. 向 URL 发送 GET 请求（跟随 HTTP 重定向）。
3. 如果响应包含 ALI 头（HTTP 头部 `x-Authlib-Injector-API-Location`），那么 ALI 指向的 URL 就是 API 地址。
 - `x-Authlib-Injector-API-Location` 可以是绝对 URL，亦可以是相对 URL。
 - 如果 ALI 指向其自身，那就意味着当前 URL 就是 API 地址。
4. 如果响应不包含 ALI 头部，则默认认为当前 URL 是 API 地址。

伪代码：

```
function resolve_api_url(url)
    response = http_get(url) // follow redirects

    if response.headers["x-authlib-injector-api-location"] exists
        new_url = to_absolute_url(response.headers["x-authlib-injector-api-
location"])
        if new_url != url
            return new_url

    // if you are going to fetch the metadata next, 'response' can be reused
    return url
```

通过拖拽设置

此方式允许用户通过[鼠标拖拽 \(DnD\)](#)来设置验证服务器。

DnD 源可以是浏览器或者其他应用程序，而 DnD 目标则是启动器。DnD 源需要展示一段文字、图片或其他内容，示意用户将此内容拖入启动器，以添加验证服务端。在此过程中，验证服务器信息从 DnD 源传输到启动器。在完成 DnD 动作后，启动器向用户确认是否要添加此验证服务器。

拖动数据

拖动数据的 MIME 类型为 `text/plain`，内容为一段 URI，其格式如下：

```
authlib-injector:yggdrasil-server:{验证服务端的 API 地址}
```

其中的 API 地址是 URI 的一个组成，应当被[编码](#)。

拖动效果为复制（`copy`）。

HTML 示例

演示页面

在需要拖动的 DOM 节点上添加 `draggable="true"`，并处理 `dragstart` 事件：

```
<span id="dndLabel" draggable="true"
ondragstart="dndLabel_dragstart(event);">example.yggdrasil.yushi.moe</span>

function dndLabel_dragstart(event) {
    let yggdrasilApiRoot = "https://example.yggdrasil.yushi.moe/";
    let uri = "authlib-injector:yggdrasil-server:" +
    encodeURIComponent(yggdrasilApiRoot);
    event.dataTransfer.setData("text/plain", uri);
    event.dataTransfer.dropEffect = "copy";
}
```

验证服务器信息的呈现

通过向 API 地址发送 GET 请求，启动器可以获取到验证服务器的元数据（[响应格式](#)），例如服务器名称。启动器可以利用这些元数据提升用户体验。

服务器名称的显示

验证服务器在 `meta` 中的 `serverName` 里指定了验证服务器的名称。当启动器需要向用户显示一个验证服务器时，便可以使用该名称。

需要注意的是，验证服务器名称可能会出现冲突，因此启动器应当提供查看验证服务器 API 地址的方法。例如，当鼠标悬浮在验证服务器名称上时，启动器在 Tooltip 中显示其 API 地址。

对非 HTTPS 验证服务器的警告

当用户尝试设置一个使用明文 HTTP 协议的验证服务器时，启动器应向用户显示醒目警告，告知用户这可能将其信息安全置于危险之中，用户的密码将会被明文传输。

账户

一个账户对应了游戏中的一个玩家，用户可以在启动时选择一个账户进行游戏。

账户、用户和角色的关系：启动器中账户的概念，与验证服务器中用户的概念**并不相同**。与启动器中的账户相对应的，是验证服务器中的角色（Profile）。验证服务器中的用户则是一个或多个角色的所有者，其在启动器中并无对应实体。

账户信息的存储

启动器通过以下三个不可变的属性来确定一个账户：

- 账户所属验证服务器
- 账户的标识（如邮箱）
 - 通常账户标识即为邮箱。但若验证服务器返回的元数据中 `feature.non_email_login` 字段为 `true`，则表明验证服务器支持使用邮箱以外的凭证登录，即账户标识可以不是邮箱。此时，启动器**不应当**期望用户输入的账户一定是邮箱，同时应注意措辞（如使用「账户」一词代替「邮箱」一词），以免造成迷惑。（详见 [Yggdrasil 服务端技术规范 § 使用角色名称登录](#)）
- 账户所对应角色的 UUID

仅当两个账户的以上三个属性都相同时，这两个账户才是相同的，其中一个属性相同并不能代表两账户相同。同一验证服务器上可以存在多个角色；多个角色可以属于同一个用户；不同验证服务器上也可以出现具有相同 UUID 的角色。因此，启动器应**同时**使用这三个属性来标识账户。

除了以上三个属性外，账户还具有以下属性：

- 令牌（accessToken 及 clientToken）
- 账户所对应角色的名称
- 用户的 ID
- 用户的属性

安全警告：

- 记住登录状态记录的是令牌，**不是**用户的密码。密码在任何时候都**不应该**被明文存储。

以上属性都是可变的。每次登录或刷新操作后，启动器都需要更新存储的账户属性。

在下面所有的登录和刷新操作中，请求中 `requestUser` 参数都为 `true`，这样启动器便能够即时更新用户 ID 和用户属性。

账户的添加

如果用户要添加一个账户，则启动器需要询问用户使用的验证服务器、用户的账户和密码。这里的验证服务器，可以是预先设置好的，可以是用户从验证服务器列表中选择的，也可以是用户即时设置的（见上文）。

此后，启动器进行以下操作：

1. 调用相应验证服务器的[登录接口](#)，其中包含用户输入的账户和密码。
2. 如果响应中 `selectedProfile` 不为空，则登录成功，使用响应中的信息更新账户属性。流程结束。
3. 如果响应中 `availableProfiles` 为空，则用户没有任何角色，触发异常。
4. 提示用户从 `availableProfiles` 中选择一个角色。
5. 调用[刷新接口](#)，其中的令牌为登录操作所返回的令牌，`selectedProfile` 为上一步中用户选择的角色。
6. 登录成功，使用刷新响应中的信息更新账户属性。

凭证有效性的确认

启动器在使用凭证前（例如启动游戏前），需要确认其有效性。如果凭证失效，则需要用户重新登录。确认凭证有效性的步骤如下：

1. 调用[验证令牌接口](#)，其中包含账户的 `accessToken` 和 `clientToken`。
2. 如果请求成功，则当前凭证有效，流程结束。否则继续执行。
3. 调用[刷新接口](#)，其中包含账户的 `accessToken` 和 `clientToken`。
4. 如果请求成功，则使用刷新响应中的信息更新账户属性，流程结束。否则继续执行。
5. 启动器要求用户重新输入密码。
6. 调用[登录接口](#)，其中包含用户的账户和上一步输入的密码。
7. 如果登录响应中 `selectedProfile` 不为空，则：
 1. 如果 `selectedProfile` 中的 `uuid` 与账户对应角色的 UUID 相同，则使用登录响应中的信息更新用户属性。流程结束。
 2. 触发异常（原账户的角色已不可用）。

8. 从 `availableProfiles` 中找出 UUID 与账户对应角色相同的角色。如果没有，则触发异常（原账户的角色已不可用）。
9. 调用[刷新接口](#)，其中令牌为登录操作返回的令牌，`selectedProfile` 为上一步中所找到的角色。
10. 登录成功，使用刷新响应中的信息更新账户属性。

账户信息的显示

启动器在显示账户时，除了账户对应角色的名称之外，还应该显示账户所属的验证服务器（[见上文](#)），防止用户混淆不同验证服务器上的同名角色。

如果启动器要显示角色皮肤，则可以调用[查询角色属性接口](#)获取角色属性，角色属性中包含了[角色的皮肤信息](#)。

启动游戏

启动器在启动游戏前需要进行以下工作：

1. （若需要）[下载 authlib-injector](#)
2. [确认凭证有效性](#)
3. [配置预获取](#)
4. [添加启动参数](#)

其中第 1、2、3 步可以并行执行，以提升启动速度。

下载 authlib-injector

启动器可以自带 `authlib-injector.jar`，也可以在启动游戏前下载一个（并缓存）。本项目提供了一个[API](#)用于下载 `authlib-injector`。

如果你的用户主要在中国大陆，我们推荐你从 [BMCLAPI 镜像](#)下载。

配置预获取

在启动前，启动器需要向 API 地址发送 GET 请求，获取 API 元数据。该元数据会在启动时被传入游戏，这样 `authlib-injector` 就不必直接请求验证服务器，进而能提升启动速度，并防止因网络故障而导致的游戏启动时崩溃。

添加启动参数

配置 authlib-injector

启动器需要添加以下 JVM 参数（应加在主类参数前）：

1. javaagent 参数：

```
-javaagent:{authlib-injector.jar 的路径}={验证服务器 API 地址}
```

2. 配置预获取：

```
-Dauthlibinjector.yggdrasil.prefetched={Base64 编码的 API 元数据}
```

下面以 example.yggdrasil.yushi.moe 为例：

- authlib-injector.jar 位于 /home/user/.launcher/authlib-injector.jar。
- 验证服务器的 API 地址为 https://example.yggdrasil.yushi.moe/。
- 向 https://example.yggdrasil.yushi.moe/ 发送 GET 请求，获取到 API 元数据：

```
{"skinDomains":["yushi.moe"],"signaturePublickey":... (省略)
```

对上面的响应进行 Base64 编码，得到：

```
eyJza2luRG9tYWluc... (省略)
```

- 故添加的 JVM 参数为：

```
-javaagent:/home/user/.launcher/authlib-injector.jar=https://example.yggdrasil.yushi.moe/
-Dauthlibinjector.yggdrasil.prefetched=eyJza2luRG9tYWluc... (省略)
```

替换参数模板

游戏版本 JSON 文件（versions/<version>/<version>.json）中规定了启动器启动游戏时使用的参数，其中部分与验证相关的参数模板应按下表进行替换：

参数模板	替换为
<code>\${auth_access_token}</code>	账户的 accessToken

参数模板	替换为
<code>\${auth_session}</code>	账户的 <code>accessToken</code>
<code>\${auth_player_name}</code>	角色的名称
<code>\${auth_uuid}</code>	角色的 UUID（无符号）
<code>\${user_type}</code>	<code>mojang</code>
<code>\${user_properties}</code>	用户的属性（JSON 格式）（若启动器不支持，可替换为 <code>{}</code> ）

在 Minecraft 服务端使用 authlib injector

获取 authlib-injector

首先你需要从[此处](#)下载最新版本的 authlib-injector。

原版服务端 / Spigot / Paper / ...

自 authlib-injector v1.2.0 以后，需要设置 `enforce-secure-profile` 为 `true`，这一点与先前版本不同！

如果您在 MC 1.19+ 上遇到聊天消息签名相关的问题，请阅读：

👉 [authlib-injector v1.2.0 升级常见问题解答 FAQ #174](#) 👉

请将服务端 `server.properties` 中的 `online-mode` 设置为 `true`。对于 1.19+ 的服务端，还需要设置 **`enforce-secure-profile` 为 `true`**。

然后在服务端的启动命令中添加以下 JVM 参数（添加的参数位于 `-jar` 之前）：

```
-javaagent:{path/to/authlib-injector.jar}={https://your-yggdrasil-api-root.com}
```

- `{path/to/authlib-injector.jar}` 表示你在[上一步](#)中下载的 JAR 文件所在的位置（相对路径、绝对路径皆可）。
- `{https://your-yggdrasil-api-root.com}` 表示验证服务器的 URL。

例如，这是原先的启动命令：

```
java -jar minecraft_server.1.12.2.jar nogui
```

假设：

- 你下载到的 authlib-injector JAR 文件名为 `authlib-injector.jar`。
- 你将其放到了与服务端 JAR `minecraft_server.1.12.2.jar` 相同的目录下。
- 验证服务器的 URL 为 `https://example.yggdrasil.yushi.moe`。

那么添加参数后的命令行应该如下：

```
java -javaagent:authlib-injector.jar=https://example.yggdrasil.yushi.moe -jar  
minecraft_server.1.12.2.jar nogui
```

BungeeCord / Velocity

如果使用 BungeeCord 或 Velocity，那么在所有服务端**以及 BungeeCord / Velocity** 上都需要加载 authlib-injector（方法见上），同时开启 `enforce-secure-chat`。但应只有 BungeeCord / Velocity 打开 `online-mode`，后端 MC 服务端应关闭 `online-mode`。

调用 Mojang 皮肤

加载 authlib-injector 后，所有皮肤默认都是从指定的验证服务器处获取的。例如：

- `/give @p minecraft:skull 1 3 {SkullOwner:"notch"}`
- （Citizens2 插件） `/npc skin notch`

这些命令获取的都是**自定义的验证服务器上**名为 `notch` 的角色的皮肤。

如果要使用 Mojang 的皮肤，则可以在角色名称后加上 `@mojang`，如：

- `/give @p minecraft:skull 1 3 {SkullOwner:"notch@mojang"}`
- `/npc skin notch@mojang`

详细说明见 [README § 参数](#) 中的 `-Dauthlibinjector.mojangNamespace` 选项。

通过代理访问 Mojang

调用 Mojang 皮肤的功能需要 MC 服务端能够访问 Mojang API。如果你的服务端要通过代理才能访问 Mojang，那么你可以在启动时添加以下 **JVM 参数**来指定代理：

```
-Dauthlibinjector.mojangProxy=socks://<host>:<port>
```

注意：

- 只有向 Mojang 查询角色信息时才会使用此代理，材质图像下载不走代理（即使是来自 Mojang 的材质）。
- 目前仅支持 SOCKS5。

签名密钥对

概述

本文主要介绍用于数字签名的密钥对。文中将使用 OpenSSL 对密钥进行操作。

验证服务器会对以下请求的响应中的角色属性进行[数字签名](#)：

- [服务端验证客户端](#)
- [查询角色属性](#)（仅当 `unsigned=false` 时才需要）

验证服务器通过[API 元数据](#)将公钥公开，以便 authlib-injector 获取。

注意：验证服务器应避免密钥的变化。如果使用多个服务端实例进行负载均衡，那么它们应该使用同一个密钥。

密钥对的生成和处理

下面对 OpenSSL 的调用都是使用标准输入和标准输出进行输入输出的。如果要使用文件，可使用参数 `-in <file>` 和 `-out <file>`。

生成私钥

密钥算法为 RSA，推荐长度为 4096 位。

```
openssl genrsa 4096
```

生成的私钥将输出到标准输出。

从私钥生成公钥

```
openssl rsa -pubout
```

私钥从标准输入读入，公钥将输出到标准输出。

获取 authlib injector

手动下载

authlib-injector 的最新版本可以直接从 authlib-injector.yushi.moe 下载。

下载 API

authlib-injector 项目提供了一组 API 用于下载 authlib-injector 构件。其 API 入口为 <https://authlib-injector.yushi.moe/>。

获取版本列表

GET /artifacts.json

响应格式：

```
{
  "latest_build_number": 最新版本的构建号,
  "artifacts": [ // 各版本信息
    {
      "build_number": 此版本的构建号,
      "version": "此版本的版本号"
    }
    // ,... (可以有更多)
  ]
}
```

获取特定版本

GET /artifact/{build_number}.json

URL 中的 {build_number} 参数代表版本构建号。

响应格式：

```
{  
  "build_number": "此版本的构建号",  
  "version": "此版本的版本号",  
  "download_url": "此版本的 authlib-injector 的下载地址",  
  "checksums": { // 校验和  
    "sha256": "SHA-256 校验和"  
  }  
}
```

获取最新版本

GET /artifact/latest.json

响应格式[同上](#)。如果你只需要获取最新版本，使用此 API 足矣。

BMCLAPI 镜像

使用 BMCLAPI 时请遵守 [BMCLAPI 的协议](#)。

BMCLAPI 为本下载 API 提供了一个[镜像](#)，其入口为
<https://bmclapi2.bangbang93.com/mirrors/authlib-injector/>。

Home

Introduction

The goals of this project are:

- To provide tools for modifying Minecraft to use custom Yggdrasil services.
- To provide technical specifications for custom Yggdrasil servers and launchers that use custom Yggdrasil services.
- To provide players with a unified non-Mojang out-of-game login experience.
 - Players can use any launcher that implements this specification to log into any Yggdrasil service that also implements this specification.

This project provides detailed documentation for all APIs and also defines several APIs that do not belong to Yggdrasil. This is done to simplify the process of specifying a Yggdrasil service as much as possible: you only need to fill in the URL corresponding to the Yggdrasil service to use it.

If you are a Yggdrasil server developer, a launcher developer, or are interested in this project, please “Watch” this project to stay informed about the latest developments in the specification.

Developer communication QQ Group: 926979364, Telegram Group: [@authlib_injector](#).
Launcher or skin server developers are welcome to join. Regular users should not join, as they may find the discussions difficult to follow.

Related Projects

- [yggdrasil-mock](#)
 - A reference implementation of the Yggdrasil server specification and test cases for the Yggdrasil API.
 - A Yggdrasil server demo site based on this project: [auth-demo.yushi.moe](#)
- [BMCLAPI](#)
 - BMCLAPI provides a mirror for downloading authlib-injector.
- [Yggdrasil API for Blessing Skin](#)
 - The Yggdrasil plugin for the Blessing Skin server.
- [HMCL](#)

- HMCL v3.x supports authlib-injector.
- [BakaXL](#)
 - BakaXL 3.0 supports authlib-injector.
- [LaunchHelper](#)
 - If you want to use authlib-injector on a Multicraft panel server, you can try this project.

Recommended Public Authentication Servers

- [LittleSkin](#)
- [Ely.by](#)

Donations

BMCLAPI provides a download mirror for authlib-injector. If you would like to support the development of authlib-injector, you can [donate to BMCLAPI](#).

Yggdrasil Server Technical Specification

Overview

This document aims to provide an unofficial technical specification for implementing a Yggdrasil server.

The server behaviors described in this specification are not necessarily identical to those of the Mojang server. This is because Mojang's server is closed source; we can only speculate on its internal logic, and such speculations will inevitably deviate from reality. However, as long as the client can correctly understand and process the server's response, whether its behavior is identical to the Mojang server is irrelevant.

Basic Conventions

Character Encoding

The character encoding used everywhere in this document is UTF-8.

Request and Response Format

Unless otherwise specified, requests and responses are in JSON format (if there is a body), and the `Content-Type` is always `application/json; charset=utf-8`.

All APIs should use the HTTPS protocol.

Error Message Format

```
{  
  "error": "Brief description of the error (Machine readable)",  
  "errorMessage": "Detailed error information (Human readable)",  
  "cause": "The cause of the error (Optional)"  
}
```

When encountering the exceptions described in this document, the returned error message should conform to the corresponding requirements.

The table below lists error messages for common exceptions. Unless otherwise specified, `cause` is generally not included.

Non-standard indicates that because it is impossible to trigger the corresponding exception on Mojang's Yggdrasil server, the error message in such cases can only be speculated.

Undefined indicates there is no explicit requirement for this item.

Exception situation	HTTP status code	Error	Error message
General HTTP Exception (Non-business exception, e.g. <i>Not Found</i> , <i>Method Not Allowed</i>)	<i>Undefined</i>	<i>The Reason Phrase corresponding to the HTTP status (defined in HTTP/1.1)</i>	<i>Undefined</i>
Invalid token	403	ForbiddenOperationException	Invalid token.
Wrong password, or login temporarily banned due to multiple failures in a short time	403	ForbiddenOperationException	Invalid credentials. Invalid username or password.
Attempting to assign a profile to a token that already has a profile assigned	400	IllegalArgumentException	Access token already has a profile assigned.
Attempting to assign a profile not belonging to the corresponding user to a token (<i>Non-standard</i>)	403	ForbiddenOperationException	<i>Undefined</i>
Attempting to join a server	403	ForbiddenOperationException	Invalid token.

Exception situation	HTTP status code	Error	Error message
using the wrong profile			

Data Format

We agree on the following data formats:

- **Unhyphenated UUID:** Refers to the UUID string with all - characters removed.

Models

User

A system can contain multiple users. A user has the following attributes:

- ID
- Email
- Password

Where ID is an unhyphenated UUID. The email can be changed, but must remain unique.

Serialization of User Information

User information conforms to the following format after serialization:

```
{
  "id": "User ID",
  "properties": [ // User attributes (Array, each element is an attribute)
    { // An attribute
      "name": "Attribute Name",
      "value": "Attribute Value",
    }
    // ,... (Can have more)
  ]
}
```

Known items in user attributes are as follows:

Name	Value
preferredLanguage	(Optional) User's preferred language, e.g. <code>en</code> , <code>zh_CN</code>

Profile

Mojang currently does not support multiple profiles; so the documentation here may not match Mojang's implementation.

Profiles and accounts have a many-to-one relationship. A profile corresponds to a player entity in Minecraft. A profile has the following attributes:

- UUID
- Name
- Texture model (either `default` or `slim`)
 - **default**: Skin with normal arm width (4px)
 - **slim**: Skin with thin arms (3px)
- Textures
 - Represented as a map whose keys are either `"SKIN"` or `"CAPE"` and whose values are URLs

The UUID and Name are globally unique, but the Name is mutable. Use of the Name as an identifier should be avoided.

Profile UUID Generation

If compatibility is not considered, the profile UUID is generally randomly generated ([Version 4](#)).

However, Minecraft uses only UUIDs to identify profiles; profiles with different UUIDs are considered different even if they have the same name. If a Minecraft server migrates from another login system (Online verification, Offline verification, or others) to this login system, and the profile's UUID changes, the profile's data will be lost. To avoid this, it must be guaranteed that for the same profile, the UUID generated by this system is the same as the UUID in the previous system.

Compatibility with Offline Verification

If the Minecraft server previously used offline verification, the profile UUID is a pure function of the profile name. If the Yggdrasil server uses this method to generate profile UUIDs, bidirectional compatibility with the offline verification system can be achieved, i.e. the server can switch between the offline verification system and this login system without losing profile data.

The code to calculate the profile UUID from the profile name is as follows (Java):

```
UUID.nameUUIDFromBytes(("OfflinePlayer:" +  
characterName).getBytes(StandardCharsets.UTF_8))
```

Implementations in other languages:

- [PHP](#)

Serialization of Profile Information

Profile information conforms to the following format after serialization:

```
{  
  "id": "Profile UUID (Unhyphenated)",  
  "name": "Profile Name",  
  "properties": [ // Profile attributes (Array, each element is an attribute)  
    (Only needed in specific cases)  
    { // An attribute  
      "name": "Attribute Name",  
      "value": "Attribute Value",  
      "signature": "Digital signature of the attribute value (Only needed in  
specific cases)"  
    }  
    // ,... (Can have more)  
  ]  
}
```

Profile attributes (`properties`) and digital signatures (`signature`) do not need to be included unless otherwise specified.

`signature` is the digital signature of the attribute value, encoded as Base64. The signature algorithm is SHA1withRSA; see [PKCS #1](#). For details on the signing key, see [Signature Key Pair](#).

Profile attributes can contain the following items:

Name	Value
textures	(Optional) Base64-encoded JSON string containing the profile's texture information. See § textures Texture Information Attribute .
uploadableTextures	(Optional) Texture types that this profile can upload. This is an attribute defined by authlib-injector. See § uploadableTextures Uploadable Texture Types

textures Texture Information Attribute

The following is the format of texture information. After Base64-encoding this JSON, it becomes the value of the `textures` profile attribute.

```
{
  "timestamp": Timestamp when this attribute value was generated (Java timestamp
format, i.e., milliseconds elapsed since 1970-01-01 00:00:00 UTC),
  "profileId": "Profile UUID (Unhyphenated)",
  "profileName": "Profile Name",
  "textures": { // Profile textures
    "Texture Type (e.g., SKIN)": { // If the profile does not have this
texture, it need not be included
      "url": "Texture URL",
      "metadata": { // Texture metadata, if none, it need not be included
        "Name": "Value"
        // ,... (Can have more)
      }
    }
    // ,... (Can have more)
  }
}
```

Currently known items in texture metadata include `model`, corresponding to the profile's texture model, with values `default` or `slim`.

uploadableTextures Uploadable Texture Types

Note: This profile attribute is defined by the authlib-injector documentation. Profile attributes returned by Mojang do not include this item. Mojang only allows users to upload skins, not capes.

Considering that not all authentication servers allow users to upload skins and capes, authlib-injector defines the `uploadableTextures` profile attribute, which indicates the types of textures the profile can upload.

The value of this attribute is a comma-separated list containing the texture types that can be uploaded. Currently, the texture types are `skin` and `cape`.

For example, if the value of the `uploadableTextures` attribute is `skin`, it means a skin can be uploaded for this profile, but not a cape; if the value is `skin,cape`, then both a skin and a cape can be uploaded.

If the `uploadableTextures` attribute does not exist, no type of texture can be uploaded for this profile.

For an introduction to the texture upload interface, please refer to [§Texture Upload](#).

Texture URL Specification

Minecraft uses the texture hash as the identifier for the texture. Whenever the client downloads a texture, it caches it locally. If a texture with the same hash is needed later, the cache will be used directly. This hash is not calculated by the client. The Yggdrasil server should calculate the texture hash in advance and use it as the filename in the texture URL, i.e., the substring starting from after the last `/` in the URL to the end. The client will directly use the URL's filename as the texture's hash.

For example, in the following URL, the hash of the texture it represents is

`e051c27e803ba15de78a1d1e83491411dff6d7fd2886da0a6c34a2161f7ca99` :

`https://yggdrasil.example.com/textures/e051c27e803ba15de78a1d1e83491411dff6d7fd2886da0a6c34a2161f7ca99`

Warning

The `Content-Type` in the texture URL response header must be `image/png`. If not specified, there is a risk of a MIME Sniffing Attack.

The server can choose the method for calculating the texture hash; it is recommended to use SHA-256 or a more secure hash algorithm. As a reference, Mojang's official method is to calculate the SHA-256 of the image file.

Security of User-Uploaded Textures

Warning

- Failure to process user-uploaded textures **may lead to Remote Code Execution**.
- Failure to check image size before reading the texture **may lead to Denial of Service attacks**.

For detailed information on this security flaw: [Unchecked user uploaded textures may lead to remote code execution #10](#)

In addition to bitmap data, PNG files can store other data. If the Yggdrasil server does not check user-uploaded textures, attackers can hide malicious code within them and distribute it to clients via the Yggdrasil server. Therefore, the Yggdrasil server **must** process user-uploaded textures to remove any data unrelated to the bitmap. Specific steps are as follows:

1. Read the size of the image in the PNG file; if it is too large, it should be rejected.

- Even a very small PNG file can store an image large enough to consume all of a computer's memory (i.e., a PNG Bomb), so never read the entire file before checking the image size.
2. Check if the image is a valid skin/cape texture.
 - Skin dimensions are multiples of 64x32 or 64x64; cape dimensions are multiples of 64x32 or 22x17. Capes with dimensions that are multiples of 22x17 are not standard-sized capes; the server needs to pad their width and height with transparent pixels to a multiple of 64x32.
 3. Re-save the image file, removing irrelevant metadata.

Implementation Hint: In Java, use `ImageReader.getWidth()` to get the dimensions without reading the entire image.

Token

Tokens and accounts have a many-to-one relationship. A token is a login credential and has a validity period. Tokens have the following attributes:

- `accessToken`
- `clientToken`
- Bound Profile
- Time of Issuance

Where `accessToken` and `clientToken` are arbitrary strings (can be unhyphenated UUIDs or JWTs). `accessToken` is randomly generated by the server, and `clientToken` is provided by the client.

Given the randomness of `accessToken`, it can serve as a primary key. `clientToken` is not unique.

The bound profile can be null. It represents the profile that can use this token to play the game.

A user can have multiple tokens simultaneously, but the server should also limit the number of tokens. When the number of tokens exceeds the limit (e.g., 10), the oldest token should be revoked before issuing a new one.

Token States

Tokens have the following three states:

- **Valid**

- A token in this state can perform various operations, such as [Server Entry Verification](#), [Refresh](#).
- Newly issued tokens (i.e., tokens issued via [Login](#), [Refresh](#)) are in this state.
- **Temporarily Invalid**
 - A token in this state has no right to perform any operation except for the [Refresh operation](#).
 - When the name of the profile bound to the token changes, the token should be marked as **Temporarily Invalid**.
 - This is to allow the launcher to refresh the token, thereby obtaining the new profile name. (See [#40](#))
 - *This state is not mandatory to implement (see below for details).*
- **Invalid**
 - A token in this state has no right to perform any operation.
 - A token is in this state after being revoked. Revocation here includes [Explicit Revocation](#), [Signout](#), revocation of the original token after [Refresh](#), and token expiration.

Token state can only change from valid to invalid, or from valid to temporarily invalid and then to invalid; this process is irreversible. The refresh operation only issues a new token and does not return the original token to a valid state.

Tokens should have an expiration time limit (e.g., 15 days). When the time elapsed since issuance exceeds this limit, the token expires.

About Temporarily Invalid State

Mojang's implementation of the temporarily invalid state is as follows: For the launcher, if a token is in a temporarily invalid state, it will refresh the token to obtain a new valid token. For the Yggdrasil server, only the last issued token is valid; other previously issued tokens are in a temporarily invalid state.

Mojang likely does this to prevent users from logging in from multiple locations simultaneously (making only the last session valid). However, in fact, even if the server does not implement the temporarily invalid state, the launcher's logic works correctly.

Of course, even if we want to implement the temporarily invalid state, we do not need to model it after Mojang's implementation. Any implementation is acceptable as long as the launcher can handle it correctly. Here is an example different from Mojang's implementation:

Take a period shorter than the token expiration limit as the dividing point between valid and temporarily invalid. If the time elapsed since issuance is within this limit, the token is valid; if it exceeds this limit but is still within the expiration limit, the token is temporarily invalid.

This approach achieves the following function: If a player logs in frequently, they do not need to enter a password except for the first login. But when they have not logged in for a long time, they need to re-enter their password.

Yggdrasil API

User Section

Login

POST /authserver/authenticate

Authenticate using a password and assign a new token.

Request Format:

```
{
  "username": "Email (or other credentials, see §Login with Profile Name)",
  "password": "Password",
  "clientToken": "clientToken of the token specified by the client (Optional)",
  "requestUser": true/false, // Whether to include user info in response,
  default false
  "agent": {
    "name": "Minecraft",
    "version": 1
  }
}
```

If the request does not include `clientToken`, the server should randomly generate an unhyphenated UUID as `clientToken`. However, note that `clientToken` can be any string; meaning any `clientToken` provided in the request is acceptable and does not have to be an unhyphenated UUID.

Regarding the profile to be bound to the token: If the user has no profiles, it is null; if the user has only one profile, it is typically bound to that profile; if the user has multiple profiles, it is typically null to allow the client to select. That is, if the bound profile is null, the client is required to perform profile selection.

Response Format:

```

{
  "accessToken": "accessToken of the token",
  "clientToken": "clientToken of the token",
  "availableProfiles": [ // List of available user profiles
    // ,... Each item is a profile (Format see $Serialization of Profile
    Information)
  ],
  "selectedProfile": {
    // ... Bound profile, if null, need not be included (Format see
    $Serialization of Profile Information)
  },
  "user": {
    // ... User information (Included only when requestUser in request is
    true, format see $Serialization of User Information)
  }
}

```

Security Tip: This API can be used for password brute-forcing and should be rate-limited. Limits should target the user, not the client IP.

Login with Profile Name

In addition to logging in with an email, the verification server can also allow users to log in using a profile name. To achieve this, the verification server needs to perform the following work:

- Set the `feature.non_email_login` field in the API metadata to true. (See [API Metadata Retrieval § Feature Options](#))
- Accept using the profile name as the `username` parameter in the [Login Interface](#).

When a user logs in using a profile name, the verification server should **automatically bind the token to the corresponding profile**, meaning `selectedProfile` in the above response should be the profile used by the user to log in.

In this case, if the user possesses multiple profiles, they can skip the operation of selecting a profile. Considering some programs do not support multiple profiles (e.g., Geyser), profile selection can also be bypassed through this method.

Refresh

POST /authserver/refresh

Revoke the original token and issue a new token.

Request Format:

```

{
  "accessToken": "accessToken of the token",
  "clientToken": "clientToken of the token (Optional)",
  "requestUser": true/false, // Whether to include user info in response,
  default false
  "selectedProfile": {
    // ... The profile to select (Optional, format see $Serialization of
    Profile Information)
  }
}

```

When `clientToken` is specified, the server should check if both `accessToken` and `clientToken` are valid; otherwise, it only needs to check `accessToken`.

The `clientToken` of the newly issued token should be the same as that of the original token.

If the request includes `selectedProfile`, then this is a profile selection operation. This operation requires the profile bound to the original token to be null, and the new token will be bound to the profile specified by `selectedProfile`. If `selectedProfile` is not included, the profile bound to the new token is the same as the original token.

The refresh operation can still be performed when the token is temporarily invalid. If the request fails, the original token remains valid.

Response Format:

```

{
  "accessToken": "accessToken of the new token",
  "clientToken": "clientToken of the new token",
  "selectedProfile": {
    // ... Profile bound to the new token, if null, need not be included
    (Format see $Serialization of Profile Information)
  },
  "user": {
    // ... User information (Included only when requestUser in request is
    true, format see $Serialization of User Information)
  }
}

```

Validate Token

POST /authserver/validate

Verify if the token is valid.

Request Format:

```
{  
  "accessToken": "accessToken of the token",  
  "clientToken": "clientToken of the token (Optional)"  
}
```

When `clientToken` is specified, the server should check if both `accessToken` and `clientToken` are valid; otherwise, it only needs to check `accessToken`.

If the token is valid, the server should return HTTP status `204 No Content`; otherwise, process it as an invalid token exception.

Revoke Token

POST `/authserver/invalidate`

Revoke the given token.

Request Format:

```
{  
  "accessToken": "accessToken of the token",  
  "clientToken": "clientToken of the token (Optional)"  
}
```

The server only needs to check `accessToken`; meaning regardless of the value of `clientToken`, it will not affect the outcome.

Regardless of whether the operation is successful, the server should return HTTP status `204 No Content`.

Signout

POST `/authserver/signout`

Revoke all tokens for the user.

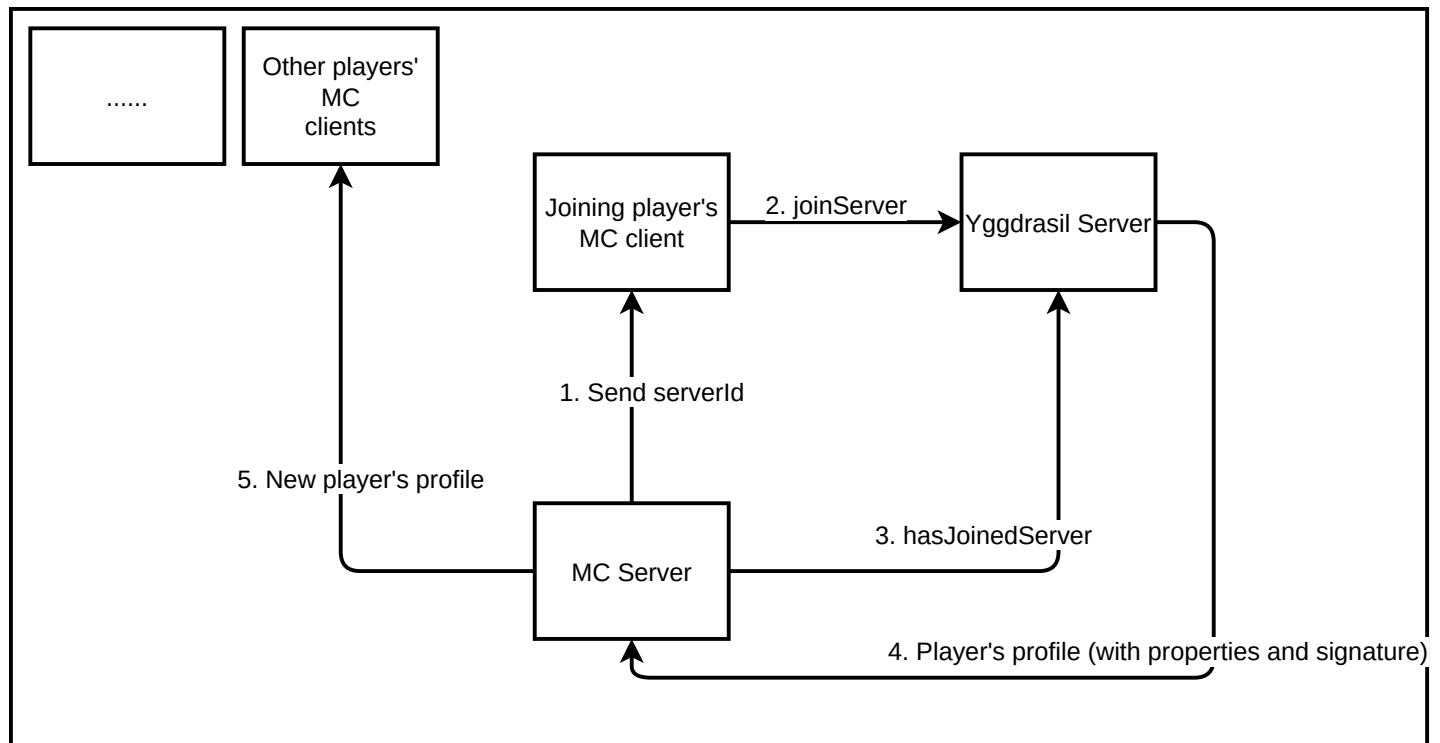
Request Format:

```
{  
  "username": "Email",  
  "password": "Password"  
}
```

If the operation is successful, the server should return HTTP status `204 No Content`.

Security Tip: This API can also be used to determine the correctness of a password, so it should be subject to the same rate limits as the Login API.

Session Section



1. The server sends a `serverId` to the joining player, which can be considered an arbitrary string.
2. The client sends the `serverId` along with its `accessToken` to the Yggdrasil server for verification.
3. The server sends the `serverId` and player's name to the Yggdrasil server to check whether verification is complete.
4. If verification succeeds, the Yggdrasil server returns the player's Profile, including texture info and digital signature.
5. The server distributes the new player's Profile to all players on the server.

The above diagram was created from [mc-server-authentication-flow.drawio](https://drawio.com).

This section is for verification when a profile joins a server. The main process is as follows:

1. The **Minecraft Server** and **Minecraft Client** jointly generate a string (`serverId`), which can be considered arbitrary.
2. The **Minecraft Client** sends the `serverId` and token to the **Yggdrasil Server** (requiring the token to be valid).

3. The **Minecraft Server** requests the **Yggdrasil Server** to check the validity of the client session, i.e. whether the client successfully completed step 2.

Client Joins Server

POST /sessionserver/session/minecraft/join

Record the `serverId` sent by the server to the client, ready for the server to check.

Request Format:

```
{
  "accessToken": "accessToken of the token",
  "selectedProfile": "UUID of the profile bound to this token (Unhyphenated)",
  "serverId": "serverId sent by the server to the client"
}
```

The operation is successful only when `accessToken` is valid and `selectedProfile` is consistent with the profile bound to the token.

The server should record the following information:

- `serverId`
- `accessToken`
- IP of the client sending this request

Note for implementation: The above information should be recorded in an in-memory database (such as Redis) and should have an expiration time set (e.g., 30 seconds). Given the randomness of `serverId`, it can serve as the primary key.

If the operation is successful, the server should return HTTP status `204 No Content`.

Server Verifies Client

GET /sessionserver/session/minecraft/hasJoined?username={username}&serverId={serverId}&ip={ip}

Check the validity of the client session, i.e., whether a record for this `serverId` exists in the database and the information is correct.

Request Parameters:

Parameter	Value
username	Name of the profile

Parameter	Value
serverId	serverId sent by the server to the client
ip (Optional)	Client IP obtained by the Minecraft Server, included only when the prevent-proxy-connections option is enabled

`username` needs to be the same as the name of the profile bound to the token corresponding to `serverId`.

Response Format:

```
{
  // ... Complete information of the profile bound to the token (Includes
  // profile attributes and digital signature, format see $Serialization of Profile
  // Information)
}
```

If the operation fails, the server should return HTTP status `204 No Content`.

Profile Section

This section is for querying profile information.

Query Profile Attributes

`GET /sessionserver/session/minecraft/profile/{uuid}?unsigned={unsigned}`

Query complete information for the specified profile (including profile attributes).

Request Parameters:

Parameter	Value
uuid	Profile UUID (Unhyphenated)
unsigned (Optional)	<code>true</code> or <code>false</code> . Whether to exclude the digital signature in the response, default is <code>true</code>

Response Format:

```
{
  // ... Profile information (Includes profile attributes. If unsigned is false,
  // digital signature must also be included. Format see $Serialization of Profile
  // Information)
}
```

If the profile does not exist, the server should return HTTP status `204 No Content`.

Batch Query Profiles by Name

`POST /api/profiles/minecraft`

Batch query profiles corresponding to profile names.

Request Format:

```
[  
  "Profile Name"  
  // ,... Can have more  
]
```

The server queries profile information corresponding to each profile name and includes it in the response. Non-existent profiles need not be included. There is no requirement for the order of profile information in the response.

Response Format:

```
[  
  {  
    // Profile information (Note: Does not include profile attributes. Format  
    see $Serialization of Profile Information)  
  }  
  // ,... (Can have more)  
]
```

Security Tip: To prevent CC attacks, a maximum value for the number of profiles in a single query needs to be set, which must be at least 2.

Texture Upload

`PUT /api/user/profile/{uuid}/{textureType}`
`DELETE /api/user/profile/{uuid}/{textureType}`

Set or clear the texture for the specified profile.

Not all profiles can upload skins and capes. To get the texture types currently uploadable by the profile, see [§ uploadableTextures](#) [Uploadable Texture Types](#).

Request Parameters:

Parameter	Value
uuid	Profile UUID (Unhyphenated)
textureType	Texture type, can be <code>skin</code> (Skin) or <code>cape</code> (Cape)

The request needs to include the HTTP header `Authorization: Bearer {accessToken}` for authentication. If the Authorization header is missing or the accessToken is invalid, return `401 Unauthorized`.

If the operation is successful, return `204 No Content`.

The usage of the two HTTP methods PUT and DELETE is described below:

PUT Upload Texture

The `Content-Type` of the request is `multipart/form-data`, and the request payload consists of the following parts:

Name	Content
model	(Only for skins) Texture model of the skin, can be <code>slim</code> (thin arm skin) or an empty string (normal skin).
file	Texture image, <code>Content-Type</code> must be <code>image/png</code> . It is suggested that the client set the <code>filename</code> parameter in <code>Content-Disposition</code> to the filename of the texture image, which can be used by the verification server as a remark for the texture.

If the operation is successful, return `204 No Content`.

DELETE Clear Texture

After clearing the texture, the texture of that type will revert to default.

Extended API

The following APIs are designed to facilitate automatic configuration for authlib-injector.

API Metadata Retrieval

GET /

Response Format:

```
{
  "meta": {
    // Server metadata, content arbitrary
  },
  "skinDomains": [ // Texture domain whitelist
    "Domain matching rule 1"
    // ,...
  ],
  "signaturePublicKey": "Public key for verifying digital signatures"
}
```

`signaturePublicKey` is a PEM format public key used to verify the digital signature of profile attributes. It starts with `-----BEGIN PUBLIC KEY-----` and ends with `-----END PUBLIC KEY-----`, allowing line breaks in between, but not other whitespace characters (line breaks at the end of the text are also allowed).

Texture Domain Whitelist

Minecraft will only download textures from domains in the whitelist. If the domain of the texture URL is not in the whitelist, a `Textures payload has been tampered with (non-whitelisted domain)` error will occur. The reason for adopting this mechanism is see [MC-78491](#).

The texture whitelist defaults to include two rules: `.minecraft.net` and `.mojang.com`. You can set the `skinDomains` attribute to add extra whitelist rules. The rule format is as follows:

- If the rule starts with `.` (dot), it matches domains ending with this rule.
 - For example, `.example.com` matches `a.example.com`, `b.a.example.com`, but **does not match** `example.com`.
- If the rule **does not start** with `.` (dot), the matched domain must be **exactly the same** as the rule.
 - For example, `example.com` matches `example.com`, but **does not match** `a.example.com`, `eexample.com`.

Metadata in meta

There is no mandatory requirement for the content in `meta`; the following fields are all optional.

Server Basic Information

Key	Value
serverName	Server Name
implementationName	Server Implementation Name
implementationVersion	Server Implementation Version

Server URLs

If you need to display the verification server homepage address, registration page address, etc., in the launcher, you can add a `links` field in `meta`.

The type of the `links` field is an object, which can contain:

Key	Value
homepage	Verification Server Homepage Address
register	Registration Page Address

Feature Options

The fields marked with **(advanced)** below are advanced options and typically **do not need** to be set.

Key	Value
feature.non_email_login	Boolean, indicates whether the verification server supports logging in with credentials other than email (such as profile name), default is false. For details, see §Login with Profile Name .
feature.legacy_skin_api	(advanced) Boolean, indicates whether the verification server supports the legacy skin API, i.e., <code>GET /skins/MinecraftSkins/{username}.png</code> . When not specified or false, authlib-injector will use the built-in HTTP server to handle requests to this API locally; if true, requests will be handled by the verification server. For details, see the <code>- Dauthlibinjector.legacySkinPolyfill</code> option in README § Parameters .

Key	Value
feature.no_mojang_namespace	(<i>advanced</i>) Boolean, whether to disable authlib-injector's Mojang namespace (@mojang suffix) feature, default is false. For details, see the – <code>Dauthlibinjector.mojangNamespace</code> option in README § Parameters.
feature.enable_mojang_anti_features	(<i>advanced</i>) Boolean, whether to enable Minecraft's anti-features, default is false. For details, see the – <code>Dauthlibinjector.mojangAntiFeatures</code> option in README § Parameters.
feature.enable_profile_key	(<i>advanced</i>) Boolean, indicates whether the verification server supports Minecraft's message signature key pair feature (i.e., digital signatures for chat messages in multiplayer games), default is false. When enabled, Minecraft will obtain the key pair issued by the verification server via the API <code>POST /minecraftservices/player/certificates</code>. When not specified or false, Minecraft will not include digital signatures in chat messages. For details, see the – <code>Dauthlibinjector.profileKey</code> option in README § Parameters.
feature.username_check	(<i>advanced</i>) Boolean, indicates whether authlib-injector enables username validation functionality, default is false. When enabled, players with special characters in their username will be unable to join the server. For details, see the – <code>Dauthlibinjector.usernameCheck</code> option in README § Parameters.

Response Example

```
{
  "meta": {
    "implementationName": "yggdrasil-mock-server",
    "implementationVersion": "0.0.1",
    "serverName": "yushijinhun's Example Authentication Server",
    "links": {
      "homepage": "https://skin.example.com/",
      "register": "https://skin.example.com/register"
    },
    "feature.non_email_login": true
  },
  "skinDomains": [
    "example.com",
    ".example.com"
  ],
  "signaturePublicKey": "-----BEGIN PUBLIC KEY-----\nMIICIj... (omitted)
...EAAQ==\n-----END PUBLIC KEY-----\n"
}
```

API Location Indication (ALI)

API Location Indication (ALI) is an HTTP response header field `X-Authlib-Injector-API-Location`, which serves the purpose of service discovery. The value of ALI is a relative URL or an absolute URL, pointing to the Yggdrasil API associated with the current page.

By using ALI, users only need to enter an address associated with the Yggdrasil API, without inputting the actual API address. For example, `https://skin.example.com/api/yggdrasil/` can be simplified to `skin.example.com`. Launchers supporting ALI will request `(https://)skin.example.com`, recognize the ALI header field in the response, and find the real API address based on it.

Skin sites can enable ALI on the homepage or site-wide. The method to enable ALI is to add the `X-Authlib-Injector-API-Location` header field to the HTTP response, for example:

```
X-Authlib-Injector-API-Location: /api/yggdrasil/ # Use relative URL
X-Authlib-Injector-API-Location: https://skin.example.com/api/yggdrasil/ #
Absolute URL is also allowed, supports cross-origin
```

When a page's ALI points to itself, this ALI will be ignored.

See Also

- [Yggdrasil - minecraft.wiki](#)
- [Mojang API - minecraft.wiki](#)
- [Protocol - minecraft.wiki](#)

Reference Implementation

[yggdrasil-mock](#) is the reference implementation for this specification.

Launcher Technical Specification

Overview

This document aims to provide technical guidance for implementing the authlib-injector specification in launchers. Since this functionality requires calling the Yggdrasil API, it is recommended that you read the [Yggdrasil Server Technical Specification](#) before reading this document.

In launchers, this login method can be referred to as “External Login (authlib-injector)” or “authlib-injector Login”. We recommend using the former, which has a clearer meaning.

Authentication Server

The authentication server (i.e., the Yggdrasil server) is the core of the entire authentication system. All authentication-related requests will be sent to it.

To identify an authentication server, the launcher should store the API address of that authentication server (i.e., the API root, such as `https://example.com/api/yggdrasil/`).

A launcher can support only one authentication server, or it can support multiple authentication servers. Supporting multiple authentication servers means that multiple accounts can exist in the launcher simultaneously, and these accounts can belong to different authentication servers.

Authentication Server Configuration

The operation of configuring an authentication server is typically performed by the player, but there are also cases where the server owner performs the configuration and distributes the configuration file along with the launcher and game. The following describes several ways to configure an authentication server:

Specifying in a Configuration File

The launcher can store the API address directly in a configuration file, allowing users to configure the authentication server by modifying the configuration file. This configuration

method is simple to implement; if your launcher is only used as a dedicated launcher for a server, you can use this configuration method.

Entering Address in Launcher

With this configuration method, the user completes the authentication server configuration by entering a URL in the launcher. This URL may be a complete API address (such as `https://example.com/api/yggdrasil/`), or it may be an abbreviated address (such as `example.com`).

When the URL does not specify a protocol (HTTPS or HTTP), we conventionally autocomplete it to the **HTTPS** protocol. In other words, `example.com/api/yggdrasil/` should be understood as `https://example.com/api/yggdrasil/`.

For security reasons, the launcher must **not** downgrade to plaintext HTTP protocol even if it cannot connect via HTTPS protocol.

Additionally, authlib-injector defines a service discovery mechanism called API Location Indication (ALI). It is used to convert abbreviated, incomplete addresses entered by users into complete API addresses.

Handling API Location Indication (ALI)

To resolve the address entered by the user to the actual API address, the launcher needs to perform the following operations:

1. If the URL is missing a protocol, autocomplete it to HTTPS protocol.
2. Send a GET request to the URL (following HTTP redirects).
3. If the response contains an ALI header (HTTP header `X-Authlib-Injector-API-Location`), then the URL pointed to by the ALI is the API address.
 - `X-Authlib-Injector-API-Location` can be an absolute URL or a relative URL.
 - If the ALI points to itself, it means the current URL is the API address.
4. If the response does not contain an ALI header, the current URL is assumed to be the API address by default.

Pseudocode:

```

function resolve_api_url(url)
    response = http_get(url) // follow redirects

    if response.headers["x-authlib-injector-api-location"] exists
        new_url = to_absolute_url(response.headers["x-authlib-injector-api-
location"])
        if new_url != url
            return new_url

    // if you are going to fetch the metadata next, 'response' can be reused
    return url

```

Configuration via Drag and Drop

This method allows users to configure the authentication server via [drag and drop \(DnD\)](#).

The DnD source can be a browser or other application, and the DnD target is the launcher. The DnD source needs to display text, an image, or other content to indicate to the user that this content should be dragged into the launcher to add an authentication server. During this process, authentication server information is transferred from the DnD source to the launcher. After completing the DnD action, the launcher asks the user to confirm whether to add this authentication server.

Drag Data

The MIME type of the drag data is `text/plain`, and the content is a URI in the following format:

```
authlib-injector:yggdrasil-server:{API address of the authentication server}
```

The API address is a component of the URI and should be [encoded](#).

The drag effect is copy (`copy`).

HTML Example

Demo Page

Add `draggable="true"` to the DOM node that needs to be dragged, and handle the `dragstart` event:

```

<span id="dndLabel" draggable="true"
ondragstart="dndLabel_dragstart(event);">example.yggdrasil.yushi.moe</span>

```

```
function dndLabel_dragstart(event) {  
    let yggdrasilApiRoot = "https://example.yggdrasil.yushi.moe/";  
    let uri = "authlib-injector:yggdrasil-server:" +  
    encodeURIComponent(yggdrasilApiRoot);  
    event.dataTransfer.setData("text/plain", uri);  
    event.dataTransfer.dropEffect = "copy";  
}
```

Authentication Server Information Display

By sending a GET request to the API address, the launcher can obtain metadata of the authentication server ([Response Format](#)), such as the server name. The launcher can use this metadata to improve user experience.

Server Name Display

The authentication server specifies its name in `serverName` within `meta`. When the launcher needs to display an authentication server to the user, it can use this name.

Note that authentication server names may conflict, so the launcher should provide a way to view the authentication server API address. For example, when the mouse hovers over the authentication server name, the launcher displays its API address in a tooltip.

Warning for Non-HTTPS Authentication Servers

When a user attempts to configure an authentication server using plaintext HTTP protocol, the launcher should display a prominent warning to the user, informing them that this may put their information security at risk and that the user's password will be transmitted in plaintext.

Account

An account corresponds to a player in the game. Users can select an account to play when launching the game.

Relationship between Account, User, and Profile: The concept of an account in the launcher is **not the same** as the concept of a user in the authentication server. What corresponds to an account in the launcher is a profile in the authentication server. A user

in the authentication server is the owner of one or more profiles and has no corresponding entity in the launcher.

Account Information Storage

The launcher identifies an account through the following three immutable attributes:

- The authentication server to which the account belongs
- The account's identifier (such as an email address)
 - Typically, the account identifier is the email address. However, if the `feature.non_email_login` field in the metadata returned by the authentication server is true, it indicates that the authentication server supports logging in with credentials other than an email address, meaning the account identifier may not be an email. In this case, the launcher should **not** assume that the account entered by the user is always an email address, and should be careful with wording (e.g., using "Account" instead of "Email") to avoid confusion. (See [Yggdrasil Server Technical Specification § Login with Profile Name](#))
- The UUID of the profile corresponding to the account

Two accounts are the same only when all three of the above attributes are the same; one attribute being the same does not mean the two accounts are the same. Multiple profiles can exist on the same authentication server; multiple profiles can belong to the same user; profiles with the same UUID can also appear on different authentication servers. Therefore, the launcher should use **all three** attributes to identify an account.

In addition to the above three attributes, an account also has the following attributes:

- Tokens (accessToken and clientToken)
- The name of the profile corresponding to the account
- The user's ID
- The user's properties

Warning

The "remember login state" feature stores tokens, **not** the user's password. The password should **never** be stored in plaintext at any time.

The above attributes are all mutable. After each login or refresh operation, the launcher needs to update the stored account attributes.

In all the login and refresh operations below, the `requestUser` parameter in the request is `true`, so that the launcher can immediately update the user ID and user properties.

Adding an Account

If a user wants to add an account, the launcher needs to ask the user for the authentication server to use, the user's account, and password. This authentication server can be pre-configured, selected by the user from a list of authentication servers, or configured by the user on the spot ([see above](#)).

Afterward, the launcher performs the following operations:

1. Call the [Login Interface](#) of the corresponding authentication server, including the account and password entered by the user.
2. If `selectedProfile` in the response is not empty, then login is successful. Use the information in the response to update the account attributes. The process ends.
3. If `availableProfiles` in the response is empty, then the user has no profiles; trigger an exception.
4. Prompt the user to select a profile from `availableProfiles`.
5. Call the [Refresh Interface](#), with the token being the token returned from the login operation, and `selectedProfile` being the profile selected by the user in the previous step.
6. Login successful. Use the information in the refresh response to update the account attributes.

Verifying Credential Validity

Before using credentials (e.g., before launching the game), the launcher needs to verify their validity. If the credentials have expired, the user needs to log in again. The steps to verify credential validity are as follows:

1. Call the [Validate Token Interface](#), including the account's `accessToken` and `clientToken`.
2. If the request succeeds, the current credentials are valid. The process ends. Otherwise, continue.
3. Call the [Refresh Interface](#), including the account's `accessToken` and `clientToken`.
4. If the request succeeds, use the information in the refresh response to update the account attributes. The process ends. Otherwise, continue.
5. The launcher asks the user to re-enter the password.
6. Call the [Login Interface](#), including the user's account and the password entered in the previous step.
7. If `selectedProfile` in the login response is not empty, then:
 1. If the `uuid` in `selectedProfile` is the same as the UUID of the profile corresponding to the account, use the information in the login response to update the user attributes. The process ends.
 2. Trigger an exception (the original account's profile is no longer available).

8. Find the profile with the same UUID as the account's corresponding profile from `availableProfiles`. If none exists, trigger an exception (the original account's profile is no longer available).
9. Call the [Refresh Interface](#), with the token being the token returned from the login operation, and `selectedProfile` being the profile found in the previous step.
10. Login successful. Use the information in the refresh response to update the account attributes.

Displaying Account Information

When displaying an account, in addition to the name of the profile corresponding to the account, the launcher should also display the authentication server to which the account belongs ([see above](#)), to prevent users from confusing profiles with the same name on different authentication servers.

If the launcher wants to display the profile skin, it can call the [Query Profile Attributes Interface](#) to obtain profile attributes, which contain the [profile's skin information](#).

Launching the Game

Before launching the game, the launcher needs to perform the following tasks:

1. (If needed) [Download authlib-injector](#)
2. [Verify Credential Validity](#)
3. [Configuration Prefetch](#)
4. [Add Launch Arguments](#)

Steps 1, 2, and 3 can be executed in parallel to improve launch speed.

Downloading authlib-injector

The launcher can include `authlib-injector.jar` itself, or it can download one (and cache it) before launching the game. This project provides [an API](#) for downloading `authlib-injector`.

If your users are primarily in mainland China, we recommend downloading from the [BMCLAPI mirror](#).

Configuration Prefetch

Before launching, the launcher needs to send a GET request to the API address to obtain API metadata. This metadata will be passed to the game at launch time, so that authlib-injector does not need to directly request the authentication server, thereby improving launch speed and preventing game crashes at startup due to network failures.

Adding Launch Arguments

Configuring authlib-injector

The launcher needs to add the following JVM arguments (which should be added before the main class argument):

1. javaagent argument:

```
-javaagent:{path to authlib-injector.jar}={authentication server API address}
```

2. Configuration prefetch:

```
-Dauthlibinjector.yggdrasil.prefetched={Base64 encoded API metadata}
```

The following uses example.yggdrasil.yushi.moe as an example:

- authlib-injector.jar is located at `/home/user/.launcher/authlib-injector.jar`.
- The authentication server API address is `https://example.yggdrasil.yushi.moe/`.
- Send a GET request to `https://example.yggdrasil.yushi.moe/` to obtain API metadata:

```
{"skinDomains":["yushi.moe"],"signaturePublicKey":... (omitted)}
```

Base64 encode the above response to get:

```
eyJza2luRG9tYWluc... (omitted)
```

- Therefore, the JVM arguments to add are:

```
-javaagent:/home/user/.launcher/authlib-injector.jar=https://example.yggdrasil.yushi.moe/
-Dauthlibinjector.yggdrasil.prefetched=eyJza2luRG9tYWluc... (omitted)
```

Replacing Parameter Templates

The game version JSON file (`versions/<version>/<version>.json`) specifies the arguments used by the launcher when launching the game. Some authentication-related parameter templates should be replaced according to the following table:

Parameter Template	Replace With
<code>\${auth_access_token}</code>	Account's accessToken
<code>\${auth_session}</code>	Account's accessToken
<code>\${auth_player_name}</code>	Profile name
<code>\${auth_uuid}</code>	Profile UUID (Unhyphenated)
<code>\${user_type}</code>	<code>mojang</code>
<code>\${user_properties}</code>	User properties (JSON format) (If the launcher does not support this, replace with <code>{}</code>)

Using authlib-injector on a Minecraft server

Obtaining authlib-injector

First, you need to download the latest version of authlib-injector from [here](#).

Vanilla server/Spigot/Paper/...

Since authlib-injector v1.2.0, you need to set `enforce-secure-profile` to `true`, which is different from previous versions!

If you encounter issues related to chat message signing on MC 1.19+, please read: [authlib-injector v1.2.0 Upgrade FAQ #174](#) 👉

Please set `online-mode` to `true` in the server's `server.properties`. For 1.19+ servers, you also need to **set `enforce-secure-profile` to `true`**.

Then add the following JVM argument to the server's startup command (the added argument should be placed **before** `-jar`):

```
-javaagent:{path/to/authlib-injector.jar}={https://your-yggdrasil-api-root.com}
```

- `{path/to/authlib-injector.jar}` indicates the location of the JAR file you downloaded in the [previous step](#) (either a relative or an absolute path is acceptable).
- `{https://your-yggdrasil-api-root.com}` indicates the URL of the authentication server.

For example, this is the original startup command:

```
java -jar minecraft_server.1.12.2.jar nogui
```

Assuming:

- The authlib-injector JAR file you downloaded is named `authlib-injector.jar`.
- You placed it in the same directory as the server JAR `minecraft_server.1.12.2.jar`.
- The authentication server URL is `https://example.yggdrasil.yushi.moe`.

Then the command line after adding the argument should be as follows:

```
java -javaagent:authlib-injector.jar=https://example.yggdrasil.yushi.moe -jar  
minecraft_server.1.12.2.jar nogui
```

BungeeCord/Velocity

If you are using BungeeCord or Velocity, then authlib-injector needs to be loaded on all servers **as well as BungeeCord/Velocity** (see the method above), and `enforce-secure-chat` should be enabled. However, only BungeeCord/Velocity should have `online-mode` enabled; the backend MC servers should have `online-mode` disabled.

Fetching Mojang Skins

After loading authlib-injector, all skins are fetched from the specified authentication server by default. For example:

- `/give @p minecraft:skull 1 3 {SkullOwner:"notch"}`
- (Citizens2 plugin) `/npc skin notch`

These commands fetch the skin of the character named `notch` from the **custom authentication server**.

If you want to use Mojang skins, you can append `@mojang` to the character name, such as:

- `/give @p minecraft:skull 1 3 {SkullOwner:"notch@mojang"}`
- `/npc skin notch@mojang`

For detailed instructions, see the `-Dauthlibinjector.mojangNamespace` option in [README § Options](#).

Accessing Mojang via Proxy

The feature to fetch Mojang skins requires the MC server to be able to access the Mojang API. If your server needs to access Mojang through a proxy, you can add the following **JVM argument** at startup to specify the proxy:

```
-Dauthlibinjector.mojangProxy=socks://<host>:<port>
```

Note:

- This proxy is only used when querying character information from Mojang; texture image downloads do not go through the proxy (even for textures from Mojang).
- Currently, only SOCKS5 is supported.

Signature Key Pair

Overview

This document introduces the key pair used for digital signatures. OpenSSL will be used to manipulate keys in this document.

The authentication server will [digitally sign](#) the profile properties in responses to the following requests:

- [Server verifies client](#)
- [Query profile attributes](#) (only required when `unsigned=false`)

The authentication server publishes its public key via [API metadata](#) so that authlib-injector can obtain it.

Note: The authentication server should avoid changing keys. If multiple server instances are used for load balancing, they should all use the same key.

Generating and Handling Key Pairs

The following OpenSSL invocations use stdin and stdout for input and output. To use files, you can use the arguments `-in <file>` and `-out <file>` .

Generating a Private Key

The key algorithm is RSA, with a recommended length of 4096 bits.

```
openssl genrsa 4096
```

The generated private key will be output to stdout.

Generating a Public Key from a Private Key

```
openssl rsa -pubout
```

The private key is read from stdin, and the public key is output to stdout.

Get authlib-injector

Manual Download

The latest version of authlib-injector can be downloaded directly from authlib-injector.yushi.moe.

Download API

The authlib-injector project provides a set of APIs for downloading authlib-injector artifacts. The API entry point is <https://authlib-injector.yushi.moe/>.

Get version list

GET /artifacts.json

Response format:

```
{
  "latest_build_number": build number of the latest version,
  "artifacts": [ // version information
    {
      "build_number": build number of this version,
      "version": "version number of this version"
    }
    // ,... (more items may follow)
  ]
}
```

Get a specific version

GET /artifact/{build_number}.json

The {build_number} parameter in the URL represents the build number of the version.

Response format:

```
{  
  "build_number": build number of this version,  
  "version": "version number of this version",  
  "download_url": "download URL of this version of authlib-injector",  
  "checksums": { // checksums  
    "sha256": "SHA-256 checksum"  
  }  
}
```

Get latest version

GET /artifact/latest.json

Response format [same as above](#). If you only need to get the latest version, this API is sufficient.

BMCLAPI Mirror

Please comply with the [BMCLAPI terms](#) when using BMCLAPI.

BMCLAPI provides a [mirror](#) for this download API, with the entry point at <https://bmclapi2.bangbang93.com/mirrors/authlib-injector/> .